ACH2003 — Computação Orientada a Objetos
Exercício Programa
Shoot 'em up

Prof. Dr. Daniel Cordeiro
Escola de Artes, Ciências e Humanidades
Universidade de São Paulo

Entrega: **26 de julho de 2026**

Descrição

Neste projeto, vocês devem reescrever o código de um jogo, disponibilizado no arquivo `Projeto_COO.zip`, de modo a aplicar diversos conceitos estudados na disciplina Computação Orientada a Objetos. Trata-se de um *shoot 'em up*[1] vertical bastante simples, sem acabamento (não possui tela de título, placar, vidas, fases, chefes, *power-ups*, etc.) e que roda de forma indefinida (até que o jogador feche a janela do jogo).

Embora funcione, o código disponibilizado **não** foi elaborado seguindo bons princípios de orientação a objetos. Apesar de escrito em Java, o código foi elaborado seguindo um estilo de programação procedural e, mesmo considerando esse estilo, não muito bem feito, uma vez que há muito código redundante. Existem, portanto, inúmeras oportunidades de melhoria do código. Os dois principais aspectos que devem ser trabalhados no desenvolvimento deste projeto são:

- A aplicação de **princípios de orientação a objetos**, através da criação de uma boa estrutura de interfaces e classes bem encapsuladas, definição de uma hierarquia de classes/interfaces adequada, e uso de recursos como herança/composição.
- O uso do **arcabouço de coleções de Java** ao invés de vetores (*arrays*) para manter/gerenciar as entidades do jogo que aparecem em multiplicidade (inimigos, projéteis, etc.).

O código do jogo é composto atualmente por apenas dois arquivos fonte: `Main.java` e `GameLib.java`. No primeiro arquivo está implementada toda a lógica do jogo, enquanto o segundo implementa uma mini biblioteca com recursos úteis no desenvolvimento de jogos: inicialização da interface gráfica, desenho de figuras geométricas simples e verificação de entrada através do teclado.

[1] `https://pt.wikipedia.org/wiki/Shoot_'em_up`

O foco da reescrita de código deve ser a classe `Main`. Pode-se assumir que a classe `GameLib` é uma biblioteca fechada à qual não se tem acesso ao código-fonte (como se realmente fosse uma biblioteca feita por terceiros) e portanto ela não precisa ser modificada neste trabalho, apenas utilizada.

Além da reescrita do código, também deverão ser implementadas algumas funcionalidades extras no jogo (se a reescrita do código for bem feita, a implementação destas funcionalidades extras deve acontecer de forma relativamente simples):

Power-ups

Itens que aparecem na tela de jogo e quando coletados pela nave do jogador melhoram algum aspecto do comportamento da nave. Devem ser implementados dois tipos de *power-ups* diferentes cujos efeitos ficam a cargo do grupo. O aparecimento dos *power-ups* deve ser definido nos arquivos de configuração das fases, como se fosse um inimigo simples, usando-se a palavra-chave `POWERUP` (mais detalhes sobre o funcionamento do mecanismo de configuração das fases são apresentados adiante).

Chefes de fase

Devem ser implementados 2 chefes de fase distintos. Os chefes de fase podem ser considerados inimigos especiais, que devem possuir as seguintes características particulares:

- Os chefes não podem sair da tela uma vez que tenham entrado na área de jogo (diferentemente dos inimigos comuns que em algum momento acabam saindo da tela caso não sejam abatidos pelo jogador);
- Os chefes devem apresentar comportamentos de ataque e movimento diferentes daqueles apresentados pelos inimigos comuns (e diferentes entre si também), além de visual próprio;
- Os chefes também devem possuir pontos de vida. Quando entrar na área de jogo uma barra de vida também deve ser exibida para o chefe;
- Deve-se assumir que um chefe aparece uma única vez durante uma fase. Além disso, a derrota de um chefe implica no avanço para a próxima fase (ver mais detalhes sobre as fases adiante).

Fases (e arquivos de configuração do jogo/fases)

Outra funcionalidade a ser implementada consiste na criação de um mecanismo para a configuração do jogo e definição de fases. Uma fase nada mais é do que um conjunto de entradas que definem *quando* e *onde* os inimigos devem aparecer, além do *tipo*. Na definição de cada fase deve-se **obrigatoriamente** incluir uma entrada que irá definir a aparição de **um** chefe. Uma fase é considerada finalizada quando o chefe é abatido pelo jogador, devendo-se então passar para a próxima fase (caso não seja a última) ou encerrar o jogo com uma mensagem de vitória (caso seja a última fase). O jogo

deve ser configurado através de um arquivo (formato texto) estruturado da seguinte forma:

```
<PONTOS DE VIDA DO JOGADOR>
<NUMERO DE FASES>
<ARQUIVO DE CONFIGURAÇÃO DA FASE 1>
<ARQUIVO DE CONFIGURAÇÃO DA FASE 2>
...
<ARQUIVO DE CONFIGURAÇÃO DA FASE N>
```

Onde: `PONTOS DE VIDA DO JOGADOR` é um valor inteiro que define a quantidade de pontos de vida com a qual o jogador começa o jogo; `NUMERO DE FASES` é um valor inteiro que define quantas fases o jogo terá; e as demais linhas definem os nomes dos arquivos de configuração de cada uma das fases (um nome de arquivo por linha, na ordem em que as fases devem ser jogadas).

Cada arquivo de configuração de fase deve estar estruturado da seguinte forma:

```
INIMIGO <TIPO> <QUANDO> <POSIÇÃO INICIAL X> <POSIÇÃO INICIAL Y>
INIMIGO <TIPO> <QUANDO> <POSIÇÃO INICIAL X> <POSIÇÃO INICIAL Y>
...
INIMIGO <TIPO> <QUANDO> <POSIÇÃO INICIAL X> <POSIÇÃO INICIAL Y>
CHEFE <TIPO> <PONTOS DE VIDA> <QUANDO> <POSIÇÃO INICIAL X> <POSIÇÃO INICIAL Y>
```

Onde: cada linha determina a aparição de um inimigo simples (definida pela palavra-chave `INIMIGO`) ou de um chefe (definida pela palavra-chave `CHEFE`). Note também que a linha que define a aparição do chefe deve ocorrer apenas uma vez no arquivo, mas esta linha não precisa ser, obrigatoriamente, a última; `TIPO` é um valor inteiro que define o tipo do inimigo ou do chefe, e deve assumir o valor 1 ou 2 (afinal, tem-se dois tipos de inimigos comuns e dois tipos de chefes, e essa numeração é independente entre as palavras-chave `INIMIGO` e `CHEFE`); `QUANDO` é um valor inteiro que define o instante em que um inimigo ou chefe deve aparecer na fase (este valor deve ser definido em milissegundos e é relativo ao instante de início da fase); `POSIÇÃO X` e `POSIÇÃO Y` são valores inteiros que definem as coordenadas do ponto no qual o inimigo ou chefe deve aparecer; e, finalmente, `PONTOS DE VIDA` é um valor inteiro que define a quantidade de pontos de vida que o chefe da fase deve possuir.

Entrega

Este Exercício-Programa deve ser feito **em duplas ou trios** e deve ser entregue na tarefa disponível no portal eDisciplinas até às 23 h 59 min do dia 26 de julho de 2026.

Você deve entregar um arquivo compactado nos formatos `.zip` ou `.tar.xz` contendo:

- um arquivo texto `LEIAME.txt`, com o nome completo e número USP de todos os integrantes do grupo e com instruções detalhadas para a **compilação** e **execução** do programa (*em linha de comando*; não dê instruções que envolvam o uso

de alguma IDE específica). Encorajamos (mas não se sintam obrigados) o uso de uma ferramenta que facilite a compilação do projeto, como o GNU Make[2] (uma ferramenta mais simples, amplamente adotada pela comunidade de desenvolvimento de código livre e usada, por exemplo, para compilar o código do próprio Linux) ou o Gradle[3] (uma ferramenta um pouco mais complexa, utilizada pela comunidade de desenvolvimento Java e adotada, por exemplo, pela comunidade Android);

- um diretório `src`, com todos os arquivos que fazem parte do código-fonte do programa reescrito e que são necessários para compilar o programa;
- um diretório `bin`, com o seu programa compilado e pronto para execução;
- um relatório **detalhado** no formato PDF contendo:
  - um ponteiro (*link*) para um vídeo de até 5 minutos demonstrando o funcionamento do jogo;
  - críticas ao código original do jogo;
  - descrição e justificativa para a nova estrutura de classes/interfaces adotada;
  - descrição de como as coleções Java foram utilizadas para substituir o uso de arrays;
  - descrição de como as novas funcionalidades foram implementadas e como o código orientado a objetos ajudou neste sentido.

Observações sobre o uso do arcabouço de coleções de Java

Na versão original do código, é feito um uso extensivo de vetores para gerenciar entidades do jogo que ocorrem em multiplicidade (inimigos, projéteis, etc.). Devido ao fato de vetores serem estruturas de armazenamento estáticas, todos os vetores são alocados com tamanhos fixos e suas posições são **reutilizadas** sempre que uma entidade associada a um determinado índice torna-se inativa (quando sai da tela, no caso dos inimigos e projéteis, ou quando é abatida pelo jogador, no caso dos inimigos).

Fazendo-se bom uso das coleções Java para armazenar e gerenciar esses conjuntos de entidades, a reutilização de posições deixa de ser necessária, pois todas as coleções implementadas pelo Java são dinâmicas. Contudo, é importante não esquecer de remover da coleção as entidades que se tornam inativas, a fim de evitar vazamentos de memória durante a execução do jogo.

Ambiente de correção

O Exercício-Programa será avaliado em um computador equipado com o sistema operacional Linux e com os seguintes softwares e versões:

[2] GNU Make: `https://www.gnu.org/software/make/`
[3] Gradle: `https://gradle.org/`

```
$ java -version
openjdk version "25.0.4-ea" 2026-07-21
OpenJDK Runtime Environment (build 25.0.4-ea+4-1-Debian)
OpenJDK 64-Bit Server VM (build 25.0.4-ea+4-1-Debian, mixed mode, sharing)

$ make --version
GNU Make 4.4.1
Compilado para x86_64-pc-linux-gnu
Copyright (C) 1988-2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

$ gradle -version
openjdk version "25.0.4-ea" 2026-07-21
OpenJDK Runtime Environment (build 25.0.4-ea+4-1-Debian)
OpenJDK 64-Bit Server VM (build 25.0.4-ea+4-1-Debian, mixed mode, sharing)

------------------------------------------------------------
Gradle 4.4.1
------------------------------------------------------------

Build time:   2012-12-21 00:00:00 UTC
Revision:     none

Groovy:       2.4.21
Ant:          Apache Ant(TM) version 1.10.17 compiled on April 13 2026
JVM:          25.0.4-ea (Debian 25.0.4-ea+4-1-Debian)
OS:           Linux 7.0.13+deb14-amd64 amd64
```

Em caso de suspeita de um problema com as versões desses softwares, o EP poderá ser executado no Linux instalado nos laboratórios didáticos de SI.

Dúvidas, problemas, etc.

Quaisquer dúvidas ou problemas relacionados ao EP devem ser discutidos no 'Fórum de discussão livre' da disciplina no eDisciplinas: `https://edisciplinas.usp.br/`.